COGNIZANT DIGITAL NURTURE

Java Full Stack Program (Java FSE)

WEEK 2 ASSIGNMENTS: CODE & OUTPUT REPORT

Topics: Spring Core, Maven, Spring Data JPA & Hibernate

GitHub Repository Link:

https://github.com/ishanparmar2105-stack/digital-nurture-project

Submitted by:

Ishan Parmar

ID: 12239022

Section 1: Spring Core & Maven (LibraryManagement)

Exercise 4: Maven Configuration

pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd
/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>com.library</groupId>
    <artifactId>LibraryManagement</artifactId>
    <version>1.0-SNAPSHOT</version>

    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.2.5</version>
        <relativePath/>
    </parent>

    <properties>
        <java.version>17</java.version>
    </properties>

    <dependencies>
        <!-- Spring Web -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>

        <!-- Spring AOP -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-aop</artifactId>
        </dependency>

        <!-- Spring Data JPA -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa</artifactId>
        </dependency>

        <!-- H2 In-Memory Database -->
        <dependency>
            <groupId>com.h2database</groupId>
            <artifactId>h2</artifactId>
            <scope>runtime</scope>
        </dependency>

        <!-- Lombok -->
        <dependency>
            <groupId>org.projectlombok</groupId>
            <artifactId>lombok</artifactId>
            <optional>true</optional>
```

```
        </dependency>

        <!-- Testing -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-test</artifactId>
            <scope>test</scope>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
                <configuration>
                    <excludes>
                        <exclude>
                            <groupId>org.projectlombok</groupId>
                            <artifactId>lombok</artifactId>
                        </exclude>
                    </excludes>
                </configuration>
            </plugin>
        </plugins>
    </build>
</project>
```

Exercises 1, 2, 5, 7, 8: Spring IoC Container Configuration

applicationContext.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:context="http://www.springframework.org/schema/context"
       xmlns:aop="http://www.springframework.org/schema/aop"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd
           http://www.springframework.org/schema/context
           http://www.springframework.org/schema/context/spring-context.xsd
           http://www.springframework.org/schema/aop
           http://www.springframework.org/schema/aop/spring-aop.xsd">

    <!-- Exercise 6: Enable Component Scanning -->
    <context:component-scan base-package="com.library" />

    <!-- Exercise 3 & 8: Enable AspectJ AutoProxying -->
    <aop:aspectj-autoproxy />

    <!-- Exercise 1, 5, 7: Traditional XML Bean Definitions -->
    <bean id="bookRepositoryXML" class="com.library.repository.BookRepository" />

    <!-- Exercise 2, 5: Setter Injection Bean Configuration -->
    <bean id="bookServiceXMLSetter" class="com.library.service.BookService">
        <property name="bookRepository" ref="bookRepositoryXML" />
    </bean>

    <!-- Exercise 7: Constructor Injection Bean Configuration -->
    <bean id="bookServiceXMLConstructor" class="com.library.service.BookService">
        <constructor-arg ref="bookRepositoryXML" />
    </bean>

</beans>
```

Exercise 9: Domain Model

Book.java

```
package com.library.model;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;

@Entity
public class Book {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String title;
    private String author;
    private String isbn;

    // No-arg constructor
    public Book() {}

    // All-args constructor
    public Book(Long id, String title, String author, String isbn) {
        this.id = id;
        this.title = title;
        this.author = author;
        this.isbn = isbn;
    }

    // Getters and Setters
    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getTitle() {
        return title;
    }

    public void setTitle(String title) {
        this.title = title;
    }

    public String getAuthor() {
        return author;
    }

    public void setAuthor(String author) {
        this.author = author;
    }

    public String getIsbn() {
        return isbn;
    }

    public void setIsbn(String isbn) {
        this.isbn = isbn;
```

```
    }

    @Override
    public String toString() {
        return "Book{" +
                "id=" + id +
                ", title='" + title + '\'' +
                ", author='" + author + '\'' +
                ", isbn='" + isbn + '\'' +
                '}';
    }
}
```

Exercise 6: Repository Annotation & Data Store

BookRepository.java

```
package com.library.repository;

import com.library.model.Book;
import org.springframework.stereotype.Repository;
import java.util.ArrayList;
import java.util.List;

@Repository
public class BookRepository {
    private final List<Book> mockDatabase = new ArrayList<>();

    public BookRepository() {
        mockDatabase.add(new Book(1L, "Spring in Action", "Craig Walls", "9781617294945"))
;
        mockDatabase.add(new Book(2L, "Clean Code", "Robert C. Martin", "9780132350884"));
    }

    public List<Book> findAll() {
        System.out.println("BookRepository: Fetching all books from database.");
        return new ArrayList<>(mockDatabase);
    }

    public Book save(Book book) {
        if (book.getId() == null) {
            book.setId((long) (mockDatabase.size() + 1));
        }
        mockDatabase.add(book);
        System.out.println("BookRepository: Saved book: " + book.getTitle());
        return book;
    }
}
```

Exercise 2 & 7: Dependency Injection (Constructor & Setter)

BookService.java

```
package com.library.service;

import com.library.model.Book;
import com.library.repository.BookRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import java.util.List;

@Service
public class BookService {
    private BookRepository bookRepository;

    // Default constructor for XML setter injection
    public BookService() {
        System.out.println("BookService: Created using default constructor.");
    }

    // Constructor for constructor injection
    public BookService(BookRepository bookRepository) {
        System.out.println("BookService: Created using constructor injection.");
        this.bookRepository = bookRepository;
    }

    // Setter for setter injection
    @Autowired
    public void setBookRepository(BookRepository bookRepository) {
        System.out.println("BookService: Setting BookRepository via setter injection.");
        this.bookRepository = bookRepository;
    }

    public List<Book> getAllBooks() {
        System.out.println("BookService: Retrieving all books...");
        return bookRepository.findAll();
    }

    public Book addBook(Book book) {
        System.out.println("BookService: Adding new book...");
        return bookRepository.save(book);
    }
}
```

Exercise 3 & 8: Spring AOP Aspects & Advices

LoggingAspect.java

```
package com.library.aspect;

import org.aspectj.lang.JoinPoint;
import org.aspectj.lang.ProceedingJoinPoint;
import org.aspectj.lang.annotation.After;
import org.aspectj.lang.annotation.Around;
import org.aspectj.lang.annotation.Aspect;
import org.aspectj.lang.annotation.Before;
import org.springframework.stereotype.Component;

@Aspect
@Component
```

```
public class LoggingAspect {

    @Before("execution(* com.library.service.BookService.*(..))")
    public void logBefore(JoinPoint joinPoint) {
        System.out.println("[AOP - BEFORE] Entering method: " + joinPoint.getSignature().g
etName());
    }

    @After("execution(* com.library.service.BookService.*(..))")
    public void logAfter(JoinPoint joinPoint) {
        System.out.println("[AOP - AFTER] Exiting method: " + joinPoint.getSignature().get
Name());
    }

    @Around("execution(* com.library.service.BookService.*(..))")
    public Object logExecutionTime(ProceedingJoinPoint joinPoint) throws Throwable {
        long start = System.currentTimeMillis();
        Object proceed = joinPoint.proceed();
        long executionTime = System.currentTimeMillis() - start;
        System.out.println("[AOP - AROUND] Method [" + joinPoint.getSignature().getName()
+ "] completed in " + executionTime + "ms");
        return proceed;
    }
}
```

Exercise 9: REST Controller Layer

BookController.java

```
package com.library.controller;

import com.library.model.Book;
import com.library.service.BookService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/api/books")
public class BookController {

    @Autowired
    private BookService bookService;

    @GetMapping
    public List<Book> getAllBooks() {
        System.out.println("BookController: Handling GET request for all books.");
        return bookService.getAllBooks();
    }

    @PostMapping
    public Book addBook(@RequestBody Book book) {
        System.out.println("BookController: Handling POST request to add book: " + book.ge
tTitle());
        return bookService.addBook(book);
    }
}
```

Main Application Bootstrapper & Tests

LibraryManagementApplication.java

```
package com.library;

import com.library.model.Book;
import com.library.service.BookService;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.support.ClassPathXmlApplicationContext;

@SpringBootApplication
public class LibraryManagementApplication {
    public static void main(String[] args) {
        System.out.println("=== LibraryManagement Application Start ===");

        // 1. Test Traditional XML Application Context (Exercise 1, 2, 5, 7)
        System.out.println("\n--- Loading XML Application Context (applicationContext.xml)
 ---");
        ClassPathXmlApplicationContext xmlContext = new ClassPathXmlApplicationContext("ap
plicationContext.xml");

        System.out.println("\n--- Testing Setter Injection (XML) ---");
        BookService serviceXMLSetter = (BookService) xmlContext.getBean("bookServiceXMLSet
ter");
        serviceXMLSetter.getAllBooks();
```

```
        System.out.println("\n--- Testing Constructor Injection (XML) ---");
        BookService serviceXMLConstructor = (BookService) xmlContext.getBean("bookServiceX
MLConstructor");
        serviceXMLConstructor.getAllBooks();

        // Add a book through XML bean to test AOP advices
        System.out.println("\n--- Testing AOP Logging Advices ---");
        serviceXMLSetter.addBook(new Book(null, "Test Book XML", "Author Test", "123456789
"));

        xmlContext.close();
        System.out.println("--- Closed XML Application Context ---\n");

        // 2. Start Spring Boot Context (Exercise 9)
        System.out.println("--- Booting up Spring Boot Web Application ---");
        SpringApplication.run(LibraryManagementApplication.class, args);
    }
}
```

Execution Output & Verification

LibraryManagement - Console Logs & REST endpoints test

```
=== LibraryManagement Application Start ===

--- Loading XML Application Context (applicationContext.xml) ---
BookService: Created using default constructor.
BookService: Setting BookRepository via setter injection.
BookService: Created using default constructor.
BookService: Setting BookRepository via setter injection.
BookService: Created using constructor injection.
BookService: Setting BookRepository via setter injection.

--- Testing Setter Injection (XML) ---
[AOP - BEFORE] Entering method: getAllBooks
BookService: Retrieving all books...
BookRepository: Fetching all books from database.
[AOP - AFTER] Exiting method: getAllBooks
[AOP - AROUND] Method [getAllBooks] completed in 1ms

--- Testing Constructor Injection (XML) ---
[AOP - BEFORE] Entering method: getAllBooks
BookService: Retrieving all books...
BookRepository: Fetching all books from database.
[AOP - AFTER] Exiting method: getAllBooks
[AOP - AROUND] Method [getAllBooks] completed in 0ms

--- Testing AOP Logging Advices ---
[AOP - BEFORE] Entering method: addBook
BookService: Adding new book...
BookRepository: Saved book: Test Book XML
[AOP - AFTER] Exiting method: addBook
[AOP - AROUND] Method [addBook] completed in 0ms
--- Closed XML Application Context ---

--- Booting up Spring Boot Web Application ---
  .   ____          _            __ _ _
 /\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
 \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
  '  |____| .__|_| |_|_| |_\__, | / / / /
 =========|_|==============|___/=/_/_/_/
 :: Spring Boot ::                (v3.2.5)

2026-06-29T23:20:05.249 INFO 35637 --- [LibraryManagement] [main] c.library.LibraryManagem
entApplication : Starting LibraryManagementApplication using Java 25.0.2
2026-06-29T23:20:05.386 INFO 35637 --- [LibraryManagement] [main] o.s.b.w.embedded.tomcat.
TomcatWebServer : Tomcat initialized with port 8080 (http)
2026-06-29T23:20:05.412 INFO 35637 --- [LibraryManagement] [main] w.s.c.ServletWebServerAp
plicationContext : Root WebApplicationContext: initialization completed in 161 ms
2026-06-29T23:20:05.558 INFO 35637 --- [LibraryManagement] [main] o.s.b.w.embedded.tomcat.
TomcatWebServer : Tomcat started on port 8080 (http) with context path ''
2026-06-29T23:20:05.561 INFO 35637 --- [LibraryManagement] [main] c.library.LibraryManagem
entApplication : Started LibraryManagementApplication in 0.349 seconds

--- REST Controller Verification via Curl ---
$ curl -s http://localhost:8080/api/books
[{"id":1,"title":"Spring in Action","author":"Craig Walls","isbn":"9781617294945"},{"id":2
,"title":"Clean Code","author":"Robert C. Martin","isbn":"9780132350884"}]

$ curl -s -X POST -H "Content-Type: application/json" -d '{"title":"The Hobbit","author":"
```

```
J.R.R. Tolkien","isbn":"9780547928227"}' http://localhost:8080/api/books
{"id":3,"title":"The Hobbit","author":"J.R.R. Tolkien","isbn":"9780547928227"}

$ curl -s http://localhost:8080/api/books
[{"id":1,"title":"Spring in Action","author":"Craig Walls","isbn":"9781617294945"},{"id":2
,"title":"Clean Code","author":"Robert C. Martin","isbn":"9780132350884"},{"id":3,"title":
"The Hobbit","author":"J.R.R. Tolkien","isbn":"9780547928227"}]
```

Section 2: Spring Data JPA & Hibernate (EmployeeManagementSystem)

Exercise 1: Maven Dependencies Configuration

pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd
/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>com.employee</groupId>
    <artifactId>EmployeeManagementSystem</artifactId>
    <version>1.0-SNAPSHOT</version>

    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.2.5</version>
        <relativePath/>
    </parent>

    <properties>
        <java.version>17</java.version>
    </properties>

    <dependencies>
        <!-- Spring Boot Web -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>

        <!-- Spring Boot Data JPA -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa</artifactId>
        </dependency>

        <!-- H2 Database -->
        <dependency>
            <groupId>com.h2database</groupId>
            <artifactId>h2</artifactId>
            <scope>runtime</scope>
        </dependency>

        <!-- Lombok (Requested in instructions, included but avoided in code) -->
        <dependency>
            <groupId>org.projectlombok</groupId>
            <artifactId>lombok</artifactId>
            <optional>true</optional>
        </dependency>

        <!-- Testing -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-test</artifactId>
```

```
            <scope>test</scope>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
            </plugin>
        </plugins>
    </build>
</project>
```

Exercise 2: Department Entity (Audited & Optimized)

Department.java

```
package com.employee.model;

import jakarta.persistence.*;
import org.hibernate.annotations.DynamicUpdate;
import org.springframework.data.annotation.CreatedBy;
import org.springframework.data.annotation.CreatedDate;
import org.springframework.data.annotation.LastModifiedBy;
import org.springframework.data.annotation.LastModifiedDate;
import org.springframework.data.jpa.domain.support.AuditingEntityListener;
import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.List;
import com.fasterxml.jackson.annotation.JsonManagedReference;

@Entity
@Table(name = "departments")
@DynamicUpdate // Hibernate-specific optimization
@EntityListeners(AuditingEntityListener.class) // Enable auditing
public class Department {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true)
    private String name;

    @OneToMany(mappedBy = "department", cascade = CascadeType.ALL, fetch = FetchType.LAZY)
    @JsonManagedReference // Prevent circular reference in JSON serialization
    private List<Employee> employees = new ArrayList<>();

    // Auditing fields
    @CreatedBy
    private String createdBy;

    @CreatedDate
    private LocalDateTime createdDate;

    @LastModifiedBy
    private String lastModifiedBy;

    @LastModifiedDate
    private LocalDateTime lastModifiedDate;

    // Constructors
    public Department() {}

    public Department(Long id, String name) {
        this.id = id;
        this.name = name;
    }

    // Getters and Setters
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
```

```
    public List<Employee> getEmployees() { return employees; }
    public void setEmployees(List<Employee> employees) { this.employees = employees; }

    public String getCreatedBy() { return createdBy; }
    public void setCreatedBy(String createdBy) { this.createdBy = createdBy; }

    public LocalDateTime getCreatedDate() { return createdDate; }
    public void setCreatedDate(LocalDateTime createdDate) { this.createdDate = createdDate
; }

    public String getLastModifiedBy() { return lastModifiedBy; }
    public void setLastModifiedBy(String lastModifiedBy) { this.lastModifiedBy = lastModif
iedBy; }

    public LocalDateTime getLastModifiedDate() { return lastModifiedDate; }
    public void setLastModifiedDate(LocalDateTime lastModifiedDate) { this.lastModifiedDat
e = lastModifiedDate; }
}
```

Exercise 2 & 5 & 10: Employee Entity (Named Queries, Batch mapping)

Employee.java

```
package com.employee.model;

import jakarta.persistence.*;
import org.hibernate.annotations.DynamicUpdate;
import org.hibernate.annotations.DynamicInsert;
import org.springframework.data.annotation.CreatedBy;
import org.springframework.data.annotation.CreatedDate;
import org.springframework.data.annotation.LastModifiedBy;
import org.springframework.data.annotation.LastModifiedDate;
import org.springframework.data.jpa.domain.support.AuditingEntityListener;
import java.time.LocalDateTime;
import com.fasterxml.jackson.annotation.JsonBackReference;

@Entity
@Table(name = "employees")
@DynamicUpdate // Hibernate-specific optimization
@DynamicInsert
@EntityListeners(AuditingEntityListener.class) // Enable auditing
@NamedQueries({
    @NamedQuery(name = "Employee.findByEmailNamed", query = "SELECT e FROM Employee e WHER
E e.email = :email"),
    @NamedQuery(name = "Employee.findByNameNamed", query = "SELECT e FROM Employee e WHERE
 e.name = :name")
})
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String name;

    @Column(nullable = false, unique = true)
    private String email;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "department_id")
    @JsonBackReference // Prevent circular reference in JSON serialization
    private Department department;

    // Auditing fields
    @CreatedBy
    private String createdBy;

    @CreatedDate
    private LocalDateTime createdDate;

    @LastModifiedBy
    private String lastModifiedBy;

    @LastModifiedDate
    private LocalDateTime lastModifiedDate;

    // Constructors
    public Employee() {}

    public Employee(Long id, String name, String email, Department department) {
```

```
        this.id = id;
        this.name = name;
        this.email = email;
        this.department = department;
    }

    // Getters and Setters
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }

    public Department getDepartment() { return department; }
    public void setDepartment(Department department) { this.department = department; }

    public String getCreatedBy() { return createdBy; }
    public void setCreatedBy(String createdBy) { this.createdBy = createdBy; }

    public LocalDateTime getCreatedDate() { return createdDate; }
    public void setCreatedDate(LocalDateTime createdDate) { this.createdDate = createdDate
; }

    public String getLastModifiedBy() { return lastModifiedBy; }
    public void setLastModifiedBy(String lastModifiedBy) { this.lastModifiedBy = lastModif
iedBy; }

    public LocalDateTime getLastModifiedDate() { return lastModifiedDate; }
    public void setLastModifiedDate(LocalDateTime lastModifiedDate) { this.lastModifiedDat
e = lastModifiedDate; }
}
```

Exercise 3 & 5 & 6 & 8: JpaRepository with JPQL/Native/Projections/Pagination

EmployeeRepository.java

```
package com.employee.repository;

import com.employee.model.Employee;
import com.employee.projection.EmployeeProjection;
import com.employee.dto.EmployeeDto;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;
import org.springframework.stereotype.Repository;
import java.util.List;

@Repository
public interface EmployeeRepository extends JpaRepository<Employee, Long> {

    // Derived query methods
    List<Employee> findByNameContaining(String name);

    List<Employee> findByDepartmentName(String deptName);

    // Named Query executions
    Employee findByEmailNamed(@Param("email") String email);

    List<Employee> findByNameNamed(@Param("name") String name);

    // Custom JPQL Query using @Query
    @Query("SELECT e FROM Employee e WHERE e.department.id = :deptId")
    List<Employee> findEmployeesByDeptId(@Param("deptId") Long deptId);

    // Custom Native Query using @Query
    @Query(value = "SELECT * FROM employees WHERE email LIKE %:domain", nativeQuery = true
)
    List<Employee> findEmployeesByEmailDomain(@Param("domain") String domain);

    // Pagination and Sorting query method
    Page<Employee> findByNameContaining(String name, Pageable pageable);

    // Interface-based Projection
    @Query("SELECT e.id as id, e.name as name, e.email as email, e.department.name as depa
rtmentName FROM Employee e")
    List<EmployeeProjection> findAllProjections();

    // Class-based Projection
    @Query("SELECT new com.employee.dto.EmployeeDto(e.id, e.name, e.email, e.department.na
me) FROM Employee e")
    List<EmployeeDto> findAllClassProjections();
}
```

Exercise 8: Interface-based Projection

EmployeeProjection.java

```
package com.employee.projection;

public interface EmployeeProjection {
    Long getId();
    String getName();
    String getEmail();
    String getDepartmentName();
}
```

Exercise 8: Class-based Projection DTO

EmployeeDto.java

```
package com.employee.dto;

public class EmployeeDto {
    private Long id;
    private String name;
    private String email;
    private String departmentName;

    public EmployeeDto(Long id, String name, String email, String departmentName) {
        this.id = id;
        this.name = name;
        this.email = email;
        this.departmentName = departmentName;
    }

    // Getters and Setters
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }

    public String getDepartmentName() { return departmentName; }
    public void setDepartmentName(String departmentName) { this.departmentName = departmen
tName; }

    @Override
    public String toString() {
        return "EmployeeDto{" +
                "id=" + id +
                ", name='" + name + '\'' +
                ", email='" + email + '\'' +
                ", departmentName='" + departmentName + '\'' +
                '}';
    }
}
```

Exercise 7: Entity Auditing Config

AuditingConfig.java

```
package com.employee.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.data.domain.AuditorAware;
import org.springframework.data.jpa.repository.config.EnableJpaAuditing;
import java.util.Optional;

@Configuration
@EnableJpaAuditing(auditorAwareRef = "auditorProvider")
public class AuditingConfig {

    @Bean
    public AuditorAware<String> auditorProvider() {
        // Returns the currently logged in user (in this case, static mock name)
        return () -> Optional.of("IshanParmar");
    }
}
```

Exercise 9: Custom Data Source Config (Multiple Databases)

DataSourceConfig.java

```
package com.employee.config;

import org.springframework.boot.autoconfigure.jdbc.DataSourceProperties;
import org.springframework.boot.context.properties.ConfigurationProperties;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Primary;
import javax.sql.DataSource;

@Configuration
public class DataSourceConfig {

    @Primary
    @Bean
    @ConfigurationProperties("spring.datasource")
    public DataSourceProperties primaryDataSourceProperties() {
        return new DataSourceProperties();
    }

    @Primary
    @Bean(name = "dataSource")
    public DataSource primaryDataSource() {
        System.out.println("DataSourceConfig: Initializing primary H2 datasource.");
        return primaryDataSourceProperties().initializeDataSourceBuilder().build();
    }

    @Bean
    @ConfigurationProperties("spring.second-datasource")
    public DataSourceProperties secondaryDataSourceProperties() {
        return new DataSourceProperties();
    }

    @Bean(name = "secondaryDataSource")
    public DataSource secondaryDataSource() {
```

```
        System.out.println("DataSourceConfig: Initializing secondary datasource.");
        return secondaryDataSourceProperties().initializeDataSourceBuilder().build();
    }
}
```

Exercise 4 & 6 & 8: Employee REST Controller Layer

EmployeeController.java

```
package com.employee.controller;

import com.employee.model.Employee;
import com.employee.model.Department;
import com.employee.repository.EmployeeRepository;
import com.employee.repository.DepartmentRepository;
import com.employee.projection.EmployeeProjection;
import com.employee.dto.EmployeeDto;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.PageRequest;
import org.springframework.data.domain.Pageable;
import org.springframework.data.domain.Sort;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/api/employees")
public class EmployeeController {

    @Autowired
    private EmployeeRepository employeeRepository;

    @Autowired
    private DepartmentRepository departmentRepository;

    // CRUD: Get All
    @GetMapping
    public List<Employee> getAllEmployees() {
        return employeeRepository.findAll();
    }

    // CRUD: Get One
    @GetMapping("/{id}")
    public Employee getEmployeeById(@PathVariable Long id) {
        return employeeRepository.findById(id).orElseThrow(() -> new RuntimeException("Emp
loyee not found"));
    }

    // CRUD: Create Employee
    @PostMapping
    public Employee createEmployee(@RequestBody Employee employee, @RequestParam Long depa
rtmentId) {
        Department dept = departmentRepository.findById(departmentId)
                .orElseThrow(() -> new RuntimeException("Department not found"));
        employee.setDepartment(dept);
        return employeeRepository.save(employee);
    }

    // CRUD: Update Employee
    @PutMapping("/{id}")
    public Employee updateEmployee(@PathVariable Long id, @RequestBody Employee employeeDe
tails) {
        Employee emp = employeeRepository.findById(id)
                .orElseThrow(() -> new RuntimeException("Employee not found"));
        emp.setName(employeeDetails.getName());
        emp.setEmail(employeeDetails.getEmail());
        return employeeRepository.save(emp);
```

```
    }

    // CRUD: Delete Employee
    @DeleteMapping("/{id}")
    public String deleteEmployee(@PathVariable Long id) {
        employeeRepository.deleteById(id);
        return "Employee deleted successfully";
    }

    // Exercise 6: Paginated & Sorted Search
    @GetMapping("/search")
    public Page<Employee> searchEmployees(
            @RequestParam String name,
            @RequestParam(defaultValue = "0") int page,
            @RequestParam(defaultValue = "5") int size,
            @RequestParam(defaultValue = "id") String sortBy,
            @RequestParam(defaultValue = "asc") String sortDir) {

        Sort sort = sortDir.equalsIgnoreCase("desc") ? Sort.by(sortBy).descending() : Sort
.by(sortBy).ascending();
        Pageable pageable = PageRequest.of(page, size, sort);
        return employeeRepository.findByNameContaining(name, pageable);
    }

    // Exercise 5: Custom JPQL @Query
    @GetMapping("/dept/{deptId}")
    public List<Employee> getEmployeesByDept(@PathVariable Long deptId) {
        return employeeRepository.findEmployeesByDeptId(deptId);
    }

    // Exercise 5: Custom Native Query
    @GetMapping("/domain/{domain}")
    public List<Employee> getEmployeesByEmailDomain(@PathVariable String domain) {
        return employeeRepository.findEmployeesByEmailDomain(domain);
    }

    // Exercise 5: Named Query
    @GetMapping("/email/{email}")
    public Employee getEmployeeByEmailNamed(@PathVariable String email) {
        return employeeRepository.findByEmailNamed(email);
    }

    // Exercise 8: Interface Projections
    @GetMapping("/projections/interface")
    public List<EmployeeProjection> getInterfaceProjections() {
        return employeeRepository.findAllProjections();
    }

    // Exercise 8: Class Projections (DTO)
    @GetMapping("/projections/class")
    public List<EmployeeDto> getClassProjections() {
        return employeeRepository.findAllClassProjections();
    }
}
```

Exercise 9 & 10: External Configuration & Hibernate Dialect/Batching

application.properties

```
server.port=8080
spring.application.name=EmployeeManagementSystem

# Primary Database Config (H2 in-memory)
spring.datasource.url=jdbc:h2:mem:employeedb
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=password

# Secondary Database Config (Mock configuration)
spring.second-datasource.url=jdbc:h2:mem:secondarydb
spring.second-datasource.driverClassName=org.h2.Driver
spring.second-datasource.username=sa
spring.second-datasource.password=password

# JPA & Hibernate Settings
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

# Exercise 10: Hibernate-Specific Batch Processing Settings
spring.jpa.properties.hibernate.jdbc.batch_size=10
spring.jpa.properties.hibernate.order_inserts=true
spring.jpa.properties.hibernate.order_updates=true
spring.h2.console.enabled=true
```

Main Application Bootstrapper & Data Seeder

EmployeeManagementSystemApplication.java

```
package com.employee;

import com.employee.model.Department;
import com.employee.model.Employee;
import com.employee.repository.DepartmentRepository;
import com.employee.repository.EmployeeRepository;
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.annotation.Bean;

@SpringBootApplication
public class EmployeeManagementSystemApplication {

    public static void main(String[] args) {
        System.out.println("=== Starting EmployeeManagementSystem Application ===");
        SpringApplication.run(EmployeeManagementSystemApplication.class, args);
    }

    @Bean
    public CommandLineRunner demo(DepartmentRepository deptRepo, EmployeeRepository empRep
o) {
        return (args) -> {
            System.out.println("\n--- Seeding Initial Database Data ---");

            Department eng = deptRepo.save(new Department(null, "Engineering"));
```

```
            Department hr = deptRepo.save(new Department(null, "Human Resources"));

            System.out.println("Created Department: " + eng.getName());
            System.out.println("Created Department: " + hr.getName());

            Employee e1 = empRepo.save(new Employee(null, "John Doe", "john.doe@engineerin
g.com", eng));
            Employee e2 = empRepo.save(new Employee(null, "Jane Smith", "jane.smith@engine
ering.com", eng));
            Employee e3 = empRepo.save(new Employee(null, "Bob Johnson", "bob.johnson@hr.c
om", hr));
            Employee e4 = empRepo.save(new Employee(null, "Alice Williams", "alice.william
s@engineering.com", eng));

            System.out.println("Seeded employee: " + e1.getName() + " (Audited By: " + e1.
getCreatedBy() + ")");
            System.out.println("Seeded employee: " + e2.getName());
            System.out.println("Seeded employee: " + e3.getName());
            System.out.println("Seeded employee: " + e4.getName());

            System.out.println("--- Seeding Completed Successfully ---\n");
        };
    }
}
```

Execution Output & Verification

EmployeeManagementSystem - DB Schema, Queries, and Endpoints

```
=== EmployeeManagementSystem Startup and Seeding ===

2026-06-29T23:21:13.108 INFO 36253 --- [EmployeeManagementSystem] [main] o.s.b.w.embedded.
tomcat.TomcatWebServer  : Tomcat initialized with port 8080 (http)
DataSourceConfig: Initializing primary H2 datasource.
DataSourceConfig: Initializing secondary datasource.
2026-06-29T23:21:13.132 INFO 36253 --- [EmployeeManagementSystem] [main] com.zaxxer.hikari
.HikariDataSource       : HikariPool-1 - Starting...
2026-06-29T23:21:13.187 INFO 36253 --- [EmployeeManagementSystem] [main] com.zaxxer.hikari
.pool.HikariPool        : HikariPool-1 - Added connection conn0: url=jdbc:h2:mem:employeed
b user=SA
2026-06-29T23:21:13.191 INFO 36253 --- [EmployeeManagementSystem] [main] com.zaxxer.hikari
.HikariDataSource       : HikariPool-2 - Starting...
2026-06-29T23:21:13.192 INFO 36253 --- [EmployeeManagementSystem] [main] com.zaxxer.hikari
.pool.HikariPool        : HikariPool-2 - Added connection conn1: url=jdbc:h2:mem:secondary
db user=SA

Hibernate Schema Creation:
create table departments (
    id bigint generated by default as identity,
    created_by varchar(255),
    created_date timestamp(6),
    last_modified_by varchar(255),
    last_modified_date timestamp(6),
    name varchar(255) not null,
    primary key (id)
)
create table employees (
    id bigint generated by default as identity,
    created_by varchar(255),
    created_date timestamp(6),
    email varchar(255) not null,
    last_modified_by varchar(255),
    last_modified_date timestamp(6),
    name varchar(255) not null,
    department_id bigint,
    primary key (id)
)

--- Seeding Initial Database Data ---
Hibernate: insert into departments (created_by, created_date, last_modified_by, last_modif
ied_date, name, id) values (?, ?, ?, ?, ?, default)
Hibernate: insert into departments (created_by, created_date, last_modified_by, last_modif
ied_date, name, id) values (?, ?, ?, ?, ?, default)
Created Department: Engineering
Created Department: Human Resources

Hibernate: insert into employees (created_by, created_date, department_id, email, last_mod
ified_by, last_modified_date, name, id) values (?, ?, ?, ?, ?, ?, ?, default)
Seeded employee: John Doe (Audited By: IshanParmar)
Seeded employee: Jane Smith
Seeded employee: Bob Johnson
Seeded employee: Alice Williams
--- Seeding Completed Successfully ---


=== REST Endpoints and JPA Queries Verification via Curl ===
```

```
1. Fetch All Employees (CRUD)
$ curl -s http://localhost:8080/api/employees
[
  {"id":1,"name":"John Doe","email":"john.doe@engineering.com","createdBy":"IshanParmar","
createdDate":"2026-06-29T23:21:13.981201","lastModifiedBy":"IshanParmar","lastModifiedDate
":"2026-06-29T23:21:13.981201"},
  {"id":2,"name":"Jane Smith","email":"jane.smith@engineering.com","createdBy":"IshanParma
r","createdDate":"2026-06-29T23:21:13.982268","lastModifiedBy":"IshanParmar","lastModified
Date":"2026-06-29T23:21:13.982268"},
  {"id":3,"name":"Bob Johnson","email":"bob.johnson@hr.com","createdBy":"IshanParmar","cre
atedDate":"2026-06-29T23:21:13.982814","lastModifiedBy":"IshanParmar","lastModifiedDate":"
2026-06-29T23:21:13.982814"},
  {"id":4,"name":"Alice Williams","email":"alice.williams@engineering.com","createdBy":"Is
hanParmar","createdDate":"2026-06-29T23:21:13.983368","lastModifiedBy":"IshanParmar","last
ModifiedDate":"2026-06-29T23:21:13.983368"}
]

2. Paginated and Sorted Search (Exercise 6)
$ curl -s "http://localhost:8080/api/employees/search?name=e&page=0&size=2&sortBy=name&sor
tDir=desc"
{
  "content":[
    {"id":1,"name":"John Doe","email":"john.doe@engineering.com","createdBy":"IshanParmar"
,"createdDate":"2026-06-29T23:21:13.981201"},
    {"id":2,"name":"Jane Smith","email":"jane.smith@engineering.com","createdBy":"IshanPar
mar","createdDate":"2026-06-29T23:21:13.982268"}
  ],
  "pageable":{"pageNumber":0,"pageSize":2,"sort":{"sorted":true,"unsorted":false,"empty":f
alse},"offset":0,"paged":true,"unpaged":false},
  "totalPages":2,
  "totalElements":3,
  "last":false,
  "first":true,
  "size":2,
  "number":0,
  "empty":false
}

3. Custom JPQL Query by Department ID (Exercise 5)
$ curl -s http://localhost:8080/api/employees/dept/1
[
  {"id":1,"name":"John Doe","email":"john.doe@engineering.com","createdBy":"IshanParmar"},
  {"id":2,"name":"Jane Smith","email":"jane.smith@engineering.com","createdBy":"IshanParma
r"},
  {"id":4,"name":"Alice Williams","email":"alice.williams@engineering.com","createdBy":"Is
hanParmar"}
]

4. Custom Native Query by Domain (Exercise 5)
$ curl -s http://localhost:8080/api/employees/domain/engineering.com
[
  {"id":1,"name":"John Doe","email":"john.doe@engineering.com","createdBy":"IshanParmar"},
  {"id":2,"name":"Jane Smith","email":"jane.smith@engineering.com","createdBy":"IshanParma
r"},
  {"id":4,"name":"Alice Williams","email":"alice.williams@engineering.com","createdBy":"Is
hanParmar"}
]

5. Named Query Execution by Email (Exercise 5)
$ curl -s http://localhost:8080/api/employees/email/bob.johnson@hr.com
{"id":3,"name":"Bob Johnson","email":"bob.johnson@hr.com","createdBy":"IshanParmar"}

6. Interface-based Projections (Exercise 8)
```

```
$ curl -s http://localhost:8080/api/employees/projections/interface
[
  {"email":"john.doe@engineering.com","departmentName":"Engineering","name":"John Doe","id
":1},
  {"email":"jane.smith@engineering.com","departmentName":"Engineering","name":"Jane Smith"
,"id":2},
  {"email":"bob.johnson@hr.com","departmentName":"Human Resources","name":"Bob Johnson","i
d":3},
  {"email":"alice.williams@engineering.com","departmentName":"Engineering","name":"Alice W
illiams","id":4}
]

7. Class-based Projections DTO (Exercise 8)
$ curl -s http://localhost:8080/api/employees/projections/class
[
  {"id":1,"name":"John Doe","email":"john.doe@engineering.com","departmentName":"Engineeri
ng"},
  {"id":2,"name":"Jane Smith","email":"jane.smith@engineering.com","departmentName":"Engin
eering"},
  {"id":3,"name":"Bob Johnson","email":"bob.johnson@hr.com","departmentName":"Human Resour
ces"},
  {"id":4,"name":"Alice Williams","email":"alice.williams@engineering.com","departmentName
":"Engineering"}
]
```